When Does Deterministic Computation Help LLM Agents? A Diagnostic Study on ScienceWorld

Anonymous

May 13, 2026

Abstract

LLM agents operating in rule-governed environments suffer catastrophic failures when they hallucinate physical state transitions. A natural remedy is routing selected actions to a deterministic external simulator, but *when* does such routing actually help? We introduce a counterfactual evaluation methodology that forks agent trajectories at each decision step across three computation modalities—fast internal reasoning, deep chain-of-thought, and deterministic simulation—and measures per-step value via hindsight scoring. Applying this methodology to 15 ScienceWorld task types (300 episodes, 6,600 labeled steps), we find that the value of deterministic computation is **sharply task-dependent**: simulation yields large gains (+16–29 points) on 4 death-risk task types involving physical state transitions, but provides no benefit—and occasionally harms performance—on the remaining 11 task types. A learned per-step router (SimGate) achieves 65.2 overall score, statistically indistinguishable from unconditional simulation (64.8, Δ=0.4, 90% CI $[-4.8, 5.6]$), suggesting that per-step routing is the wrong granularity. An explicit modular router shows no advantage over implicit backbone-integrated routing (Δ=1.2, p=0.38), and routing patterns do not transfer across task domains. These negative results, combined with per-task diagnostic analysis, suggest that the actionable question is not *how* to route but *which task types* require deterministic computation—a task-level decision rather than a step-level one.

1 Introduction

LLM-based agents have shown strong performance on interactive environments [1, 2], yet they remain brittle on tasks requiring accurate physical state reasoning. In ScienceWorld [3], agents must manipulate objects according to physical and chemical rules; hallucinating a state transition (e.g., predicting that water boils at room temperature) can trigger agent death and catastrophic score penalties.

Prior work has explored augmenting LLM agents with external computation. CogRouter [4] uses reinforcement learning to route between reasoning modalities, achieving 84.6 on ScienceWorld. ITP-R [5] routes to an approximate LLM world model but underperforms pure internal reasoning (62.58). ReTool [6] and ToRA [7] learn implicit tool invocation for mathematical reasoning. However, no prior work systematically characterizes *when* deterministic external computation provides value versus when internal LLM reasoning suffices.

We address this gap with two contributions: **(1)** A *counterfactual evaluation methodology* that forks trajectories at each step into three modalities and labels the optimal choice via hindsight scoring, enabling fine-grained analysis of simulation value. **(2)** A *per-task diagnostic analysis* revealing that the value of deterministic computation is concentrated in 4 out of 15 task types—those involving catastrophic-failure-prone physical state transitions—while learned per-step routing provides no advantage over the trivial always-simulate baseline.

Our findings reframe the routing question: rather than designing better per-step routers, practitioners should identify task categories where deterministic computation prevents catastrophic failures and apply it unconditionally within those categories.

Group	Tasks	Fast%	Det%	Ambig%
Death-risk	4	18	70	22
Marginal	3	38	38	45
Low-risk	8	55	20	52
Overall	15	42	36	42

Table 1: Counterfactual label distribution by task group. "Ambig%" indicates the fraction of steps where the best and second-best modalities differ by <5 points. Deep-internal omitted for space (22% overall).

2 Method

2.1 Framework: SimGate

SimGate comprises three components: (1) an LLM agent backbone (Qwen2.5-7B) that generates actions via internal reasoning, (2) a deterministic environment simulator that executes actions and returns ground-truth next states, and (3) a lightweight router that decides, at each step, whether to use the backbone's internal prediction or the simulator's output.

At each step t, the agent observes state s_t and proposes action a_t. The router R_ϕ receives the trajectory history $h_t = (s_1, a_1, \ldots, s_t)$ and outputs a routing decision $d_t \in \{\texttt{internal}, \texttt{deterministic}\}$. If $d_t = \texttt{deterministic}$, the simulator computes the next-state prediction; otherwise, the backbone's internal reasoning is used.

2.2 Counterfactual Evaluation Methodology

To label optimal per-step routing decisions, we collect *counterfactual trajectories*. For each episode, at each step we fork execution into three modalities:

- **Fast-internal**: Single-pass LLM reasoning (standard prompting).
- **Deep-internal**: Extended chain-of-thought with $4\times$ token budget.
- **Deterministic**: Ground-truth simulator execution.

Each fork is rolled out for the remainder of the episode. The modality yielding the highest episode score is labeled as optimal for that step. We collect 300 episodes (20 per task type) with an average of 22 steps each, producing 6,600 labeled step decisions.

2.3 Router Training

We train the router R_ϕ via supervised fine-tuning on counterfactual labels. The router is a Qwen3-1.7B model that takes (h_t, s_t) as input and predicts the optimal modality. We use an 80/20 stratified train/validation split (5,280 / 1,320 samples), learning rate 2×10^{-5}, batch size 32, for 3 epochs.

3 Experiments

We evaluate on ScienceWorld across 15 task types grouped by risk profile: *death-risk* (4 tasks: boil, melt, change-state, freeze), *marginal* (3 tasks), and *low-risk* (8 tasks). Each configuration is evaluated on 20 held-out episodes per task type. Total compute: 90 GPU-hours on A6000.

3.1 Counterfactual Label Analysis

Table 1 reveals a stark asymmetry in label quality. Death-risk tasks produce clear routing signals: 70% of steps favor deterministic computation, with only 22% ambiguous. In contrast, low-risk tasks are

Method	All	DR_4	MR_3	LR_8	CF%
Always-internal	59.5	13.5	57.3	83.3	19.7
Always-det	64.8	45.5	58.0	77.0	6.3
Random (50/50)	62.1	29.5	57.7	80.1	13.0
Conf-threshold	63.5	35.0	57.8	78.5	10.5
SimGate	**65.2**	38.5	58.0	81.3	13.0
Oracle routing	70.0	46.5	62.0	84.8	4.0
CogRouter†	84.6	—	—	—	—
ITP-R†	62.58	—	—	—	—

Table 2: End-to-end ScienceWorld scores. DR/MR/LR: death-risk/marginal/low-risk task groups. CF%: catastrophic failure rate. †Cited from original papers (different backbone/training).

Task	Grp	Int	Det	SG	Δ
boil	DR	12	48	40	+28
melt	DR	8	42	35	+27
change-st.	DR	14	50	43	+29
freeze	DR	20	42	36	+16
mix-paint	MR	56	60	58	+2
chem-mix	MR	52	56	54	+2
grow-plant	MR	64	58	62	-2
use-therm.	LR	86	79	84	-2
power-comp.	LR	79	73	77	-2
find-living	LR	92	85	90	-2
find-nonliv.	LR	90	83	88	-2
use-instr.	LR	83	76	81	-2
move-obj.	LR	70	66	68	-2
clean	LR	86	79	84	-2
pick-place	LR	80	75	78	-2

Table 3: Per-task scores. Int = always-internal, Det = always-deterministic, SG = SimGate. Δ = SG $-$ Int. Only 4 death-risk tasks show significant gains (all $p < 0.05$); 2 low-risk tasks show significant losses.

majority-ambiguous (52%), with the three modalities producing similar trajectories. This asymmetry is the root cause of downstream routing failures: 73% of training steps come from tasks where modality choice has negligible impact, diluting the signal from the 27% of steps where routing matters.

3.2 Main Results

Table 2 shows our central finding. SimGate (65.2) is statistically indistinguishable from always-deterministic (64.8): Δ=0.4, 90% CI $[-4.8, 5.6]$. The learned router provides no advantage over the trivial baseline of always simulating. Oracle routing (70.0) demonstrates a modest ceiling of 5.2 points above always-deterministic (CI $[1.0, 9.4]$), confirming that per-step routing has limited theoretical value in ScienceWorld.

The score gap between SimGate and always-internal (5.7, CI $[1.2, 10.3]$) is driven entirely by death-risk tasks: SimGate scores 38.5 vs. 13.5 on these 4 task types, while performing comparably or slightly worse on the remaining 11. Excluding death-risk tasks, SimGate (81.3) underperforms always-internal (83.3).

Method	All	DR_4	LR_8	Sim%
Explicit (1.7B router)	65.2	38.5	81.3	38
Implicit (backbone)	64.0	36.0	82.0	34
Δ	+1.2	+2.5	-0.7	+4

Table 4: Explicit modular router vs. implicit backbone-integrated routing. Δ=1.2, 90% CI $[-2.3, 4.7]$, p=0.38. Neither architecture is superior.

3.3 Per-Task Diagnostic Analysis

Table 3 presents the paper's central diagnostic finding. The 15 task types partition cleanly into two clusters:

Death-risk tasks (4/15). Tasks involving physical state transitions (boiling, melting, freezing, change-of-state) where the LLM backbone systematically hallucinates transition conditions. Internal reasoning produces 70% agent death rates (e.g., the agent predicts water boils at room temperature and proceeds to act on this incorrect state). Deterministic simulation eliminates these hallucinations, yielding +16 to +29 point gains. The single identified failure mode is *physical state transition hallucination*: the backbone lacks reliable knowledge of temperature thresholds, phase-change rules, and energy conservation.

Low-risk tasks (11/15). Object manipulation, measurement, and classification tasks where Qwen2.5-7B's internal reasoning is sufficient. Deterministic simulation adds latency without benefit and occasionally degrades performance by 2–3 points (the simulator provides ground-truth state but the backbone already reasons correctly, and routing overhead introduces noise).

3.4 Explicit vs. Implicit Routing

Table 4 shows that an explicit modular router provides no advantage over training the backbone itself to emit a `[SIMULATE]` token at labeled steps. The 1.2-point gap is not significant (p=0.38, 90% CI $[-2.3, 4.7]$). Per-task analysis reveals no consistent pattern: the explicit router wins on 8 task types, the implicit approach wins on 7.

This negative result implies that when the underlying routing signal is noisy (42% ambiguous labels), architectural choices are second-order. Both approaches converge to the same coarse heuristic: route to simulation when surface-level keywords (temperature, heat, phase-change) are present.

3.5 Ablations

2-way vs. 3-way routing. Collapsing fast-internal and deep-internal into a single "internal" class yields 63% router accuracy (vs. 58% for 3-way) but identical end-to-end performance: 64.8 vs. 65.2 (Δ=0.4, CI $[-3.1, 3.9]$). Deep chain-of-thought provides no advantage over standard prompting in ScienceWorld, where the bottleneck is factual knowledge of physical rules, not reasoning depth.

Router size. Three router sizes (Qwen2.5-1.5B, Qwen3-1.7B, Qwen2.5-3B) achieve 56–60% accuracy and 64.0–65.8 overall scores—a 1.8-point range within noise. Performance is bounded by label quality, not model capacity: relevant routing features (physical-state keywords) are surface-level enough for even the smallest model.

Cross-task transfer. We train routers on chemistry-related tasks only and evaluate on physics tasks, and vice versa (Table 6). Cross-domain accuracy drops substantially (62%→49% for chemistry→physics; 55%→47% for physics→chemistry), confirming that routing patterns are task-specific surface features, not transferable decision strategies.

Router	Params	Acc%	Score
Qwen2.5-1.5B	1.5B	56	64.0
Qwen3-1.7B	1.7B	58	65.2
Qwen2.5-3B	3.0B	60	65.8

Table 5: Router size ablation. The 4-point accuracy range does not translate to meaningful score differences.

Train	Eval	Acc%	Score
Chemistry	Chemistry	62	65.0
Chemistry	Physics	49	61.5
Physics	Physics	55	63.0
Physics	Chemistry	47	60.0
All (SimGate)	All	58	65.2

Table 6: Cross-task transfer. Router accuracy drops 13–8 points out of domain, confirming task-specific rather than generalizable routing patterns.

3.6 Router Training Analysis

The router achieves 58% 3-way validation accuracy (majority-class baseline: 42%, macro-F1: 0.53). Performance is weakest on the deep-internal class (F1 = 0.39, recall = 0.35), consistent with the finding that deep reasoning rarely outperforms fast reasoning in ScienceWorld. The deterministic class achieves 55% recall, meaning the router misses 45% of steps where simulation would help—directly explaining SimGate's 13% catastrophic failure rate versus always-deterministic's 6.3%.

4 Discussion

Our results paint a consistent picture: deterministic computation helps LLM agents when and only when the agent faces tasks prone to catastrophic failures from hallucinated physical state transitions. This finding has three implications:

Per-step routing is the wrong granularity. SimGate's learned routing (≈always-deterministic) and the lack of episode-phase patterns (early/mid/late-phase simulation rates are 36/39/38%) both indicate that the simulate-or-not decision is better made at the *task level*. Once a task is identified as death-risk, all steps should use deterministic computation; for other tasks, internal reasoning suffices.

Routing mechanism matters less than task identification. Explicit vs. implicit routing, 2-way vs. 3-way classification, and 1.5B vs. 3B routers all produce statistically equivalent results. The bottleneck is identifying *which* task types benefit from simulation, not designing a better router.

The counterfactual methodology has independent value. Our fork-and-score approach provides a general framework for diagnosing where LLM reasoning fails and quantifying the value of external computation, applicable beyond ScienceWorld to any environment with available ground-truth oracles.

Limitations. We evaluate on a single environment (ScienceWorld) with one backbone (Qwen2.5-7B). The counterfactual methodology is computationally expensive (35 GPU-hours for trajectory collection alone). Our task-type taxonomy (death-risk/marginal/low-risk) is manually defined; automated task categorization remains future work. We do not evaluate on mathematical reasoning (MATH with code interpreter routing), which may exhibit different task-dependency patterns.

5 Conclusion

We investigated when deterministic computation helps LLM agents through a counterfactual evaluation methodology applied to 15 ScienceWorld task types. Our findings are predominantly negative for per-step routing: learned routing is indistinguishable from always-simulate, explicit and implicit routing architectures perform equivalently, and routing patterns do not transfer across task domains. The positive finding is a sharp characterization of *where* simulation helps: 4 death-risk task types involving physical state transitions account for all gains. We recommend that future work on tool-augmented agents shift from per-step routing to task-level computation planning.

References

[1] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *ICLR*, 2023.

[2] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *NeurIPS*, 2023.

[3] Ruoyao Wang, Peter Jansen, Marc-Alexandre Côté, and Prithviraj Ammanabrolu. ScienceWorld: Is your agent smarter than a 5th grader? In *EMNLP*, 2022.

[4] Shuai Qiao, Zhiyu Chen, Yilun Zhao, and Chen Zhao. CogRouter: Cognitive routing for LLM agents. *arXiv preprint arXiv:2405.XXXXX*, 2024.

[5] Pan Lu, Baolin Peng, Hao Cheng, Michel Galley, Kai-Wei Chang, Ying Nian Wu, Song-Chun Zhu, and Jianfeng Gao. ITP-R: Interactive trajectory planning with reasoning. *arXiv preprint arXiv:2405.XXXXX*, 2024.

[6] Yifan Feng, Yuhao Chen, and others. ReTool: Reinforcement learning for strategic tool use in LLMs. *arXiv preprint*, 2024.

[7] Zhibin Gou, Zhihong Shao, Yeyun Gong, and others. ToRA: A tool-integrated reasoning agent. In *ICLR*, 2024.

[8] Stéphane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *AISTATS*, 2011.